Registro de Decisiones de Arquitectura (ADR)

Plataforma de Farmacología Gamificada — Lote 3 (ADR-0007 a ADR-0010)

Versión 1 — OFICIAL

Fecha del documento: 23 de septiembre de 2026

ADR-0007: Elección de ORM: Prisma

Fecha: 23 de septiembre de 2026

Estado: Aceptada

Contexto

Adrián tiene poca experiencia previa en Node.js y necesita minimizar el riesgo de errores de seguridad al acceder a MySQL (ej. inyección SQL), manteniendo buena productividad para un equipo de 1-2 personas.

Opciones consideradas

- SQL puro con un driver como mysql2, construyendo las consultas manualmente.
- Prisma como ORM, con generación de cliente TypeScript y migraciones.
- Sequelize como ORM alternativo.

Decisión

Se adopta Prisma como ORM para todo el acceso a MySQL en el monolito.

Trade-offs

Aspecto	Análisis
A favor	Genera consultas parametrizadas automáticamente (mitiga inyección SQL); tipado fuerte con TypeScript, reduciendo errores; sistema de migraciones que mantiene sincronizada la base de datos con el código; curva de aprendizaje razonable para alguien sin mucha experiencia previa.
En contra / riesgos	Añade una capa de abstracción que oculta el SQL real, lo cual puede dificultar optimizar consultas muy específicas en el futuro; requiere aprender su propio lenguaje de esquema (Prisma Schema).
Alternativa descartada	SQL puro da control total pero aumenta el riesgo de errores de seguridad y de sintaxis para alguien con poca experiencia; Sequelize es válido pero tiene menos integración con TypeScript que Prisma.

ADR-0008: Estrategia de autenticación: JWT sin estado

Fecha: 23 de septiembre de 2026

Estado: Aceptada

Contexto

La plataforma maneja suscripciones pagas pero no datos bancarios ni médicos sensibles. Se necesita un mecanismo de autenticación seguro pero proporcional al riesgo real, compatible con la decisión de backend stateless para escalabilidad (ADR-0003).

Opciones consideradas

- Sesión tradicional almacenada en el servidor (con estado).
- JWT (JSON Web Token) sin estado, con contraseñas cifradas con bcrypt.
- Autenticación multifactor obligatoria con proveedores externos de identidad (nivel bancario).

Decisión

Se adopta JWT sin estado para las sesiones, con contraseñas almacenadas usando bcrypt (hashing con salt). No se implementa autenticación multifactor obligatoria ni infraestructura de nivel bancario, dado que no se manejan datos financieros ni médicos sensibles.

Trade-offs

Aspecto	Análisis
A favor	Compatible con el backend stateless (ADR-0003), permitiendo escalar horizontalmente sin problemas de sesión compartida; bcrypt es un estándar probado y suficiente para este nivel de riesgo; JWT es simple de integrar en un monolito Node.js.
En contra / riesgos	Los JWT deben manejarse con cuidado (expiración adecuada, no guardarlos en lugares inseguros del navegador) para evitar robo de sesión; no incluye multifactor desde el inicio.
Alternativa descartada	Un esquema de seguridad de nivel bancario (multifactor obligatorio, infraestructura dedicada) sería sobre-ingeniería para el riesgo real de esta plataforma y ralentizaría el desarrollo sin beneficio proporcional.

ADR-0009: Validación de datos de entrada con Zod

Fecha: 23 de septiembre de 2026

Estado: Aceptada

Contexto

Los módulos se comunican mediante DTOs (ADR-0004) y reciben datos de usuarios a través de la API. Se necesita validar esa información de forma consistente antes de que llegue a la lógica de negocio.

Opciones consideradas

- Validación manual con condicionales en cada endpoint.
- Joi como librería de validación.
- Zod como librería de validación.

Decisión

Se adopta Zod para validar los datos de entrada en los DTOs de todos los módulos.

Trade-offs

Aspecto	Análisis
A favor	Integración nativa con TypeScript (infiere tipos automáticamente desde el esquema de validación); reduce código repetitivo comparado con validación manual; ampliamente usado en proyectos modernos de Node.js/TypeScript.
En contra / riesgos	Requiere aprender su sintaxis específica; otra dependencia más a mantener actualizada.
Alternativa descartada	La validación manual es propensa a errores y omisiones; Joi es una alternativa válida pero con menor integración directa con TypeScript que Zod.

ADR-0010: Reglas de progreso, evaluación final y certificación

Fecha: 23 de septiembre de 2026

Estado: Aceptada

Contexto

Se necesita definir formalmente cómo avanza un estudiante dentro de un Tema, qué significa completar una lección, y qué sucede al terminar un Tema completo, ya que esto impacta directamente el modelo de datos de los módulos de Contenido, Desafíos y Gamificación.

Opciones consideradas

- Todas las lecciones desbloqueadas desde el inicio, sin orden obligatorio.
- Desbloqueo progresivo de lecciones a medida que se completan las anteriores dentro del tema, con examen general y diploma al finalizar el tema completo.

Decisión

Se adoptan las siguientes reglas: (1) una lección se completa cuando el estudiante responde/completa todos sus desafíos asociados; (2) las lecciones de un tema se desbloquean progresivamente, en orden, a medida que se completan las anteriores; (3) al completar todas las lecciones de un Tema, se genera un examen general de todo el tema y, al aprobarlo, un diploma del curso; (4) el estudiante puede repasar libremente cualquier tema ya completado, sin restricciones; (5) repetir un desafío ya completado no otorga XP adicional, solo la primera vez se otorga XP.

Trade-offs

Aspecto	Análisis
A favor	El desbloqueo progresivo refuerza el aprendizaje ordenado (menos a más) ya definido en los requisitos; el examen general y el diploma dan un cierre motivador por tema, reforzando el aspecto gamificado; evitar XP repetido previene que los estudiantes "farmeen" XP artificialmente rehaciendo el mismo desafío.
En contra / riesgos	El examen general y el diploma introducen nuevas entidades de datos (examen, resultado del examen, diploma emitido) que deben añadirse al modelo de base de datos; el desbloqueo progresivo requiere lógica adicional para calcular qué lección está disponible para cada estudiante.
Alternativa descartada	Dejar todas las lecciones abiertas desde el inicio es más simple de implementar, pero no refuerza la progresión pedagógica de menos a más que se definió como requisito del proyecto.